

# Assignment Report

The Assignment report submitted in partial fulfilment of the

requirements for

the award of the degree of

## The Professional Master Of Embedded And Cyber-Physical Systems



### ECPS 216 - F'24: IOT Systems & Software

#### Programming Assignment 2- (Final Version)

{Raspberry Pi and 1 ESP8266 Swarm Communication Using LED Indicators and Light Sensors}

Under the guidance of

Prof. Quoc-Viet Dang

Name of Candidates

Sahil Kakadiya

UCI Student ID

93282494

# Abstract

In this project, we integrate a Raspberry Pi and an ESP8266 module to demonstrate wireless communication using UDP protocol. The Raspberry Pi controls LEDs and establishes communication with the ESP8266, which is connected to a photoresistor. The Raspberry Pi initiates communication via a button press, causing the ESP8266 to collect and transmit light sensor data. The Raspberry Pi then processes the sensor data to control the RGB LEDs based on predefined light intensity thresholds. This report details the hardware setup, software design, and communication protocol, providing flowcharts and code explanations for both devices. The project simulates a functional, interactive swarm system with error handling capabilities and state resets.

# Introduction

This project extends the use of a Raspberry Pi and ESP8266 to build a simple swarm communication system using a button, LEDs, and a light sensor. The Raspberry Pi serves as the central controller, initiating communication and processing sensor data received from the ESP8266. The ESP8266 collects ambient light values through a photoresistor, sending these values back to the Raspberry Pi over WiFi. Depending on the light intensity, the Raspberry Pi changes the state of RGB LEDs to indicate low, medium, or high light levels. The project emphasizes network communication using UDP packets, error handling, and resets triggered by the user. This system demonstrates the basics of IoT communication and control, highlighting the role of real-time sensor data processing in distributed systems.

# Part 1- Raspberry Pi Wi-Fi Setup and Packet Delivery

## Raspberry Pi Code Functionality:

- **Input:** Button press to start communication, Wi-Fi status check, sensor data received from ESP8266.
- **Process:** Handling Wi-Fi connection, sending/receiving UDP messages, processing sensor data, handling different intensity levels.
- **Output:** Control LEDs based on sensor data, send start/stop messages to ESP8266, indicate errors with LED blinking.

## Flowchart for Raspberry Pi Program:

The following states and transitions represent the Raspberry Pi's functionality:

### 1. Initial State (IDLE):

- **Action:** Check **Wi-Fi** connection. If not connected, attempt reconnection.
  - **Transition:** If **Wi-Fi** is connected, move to **WIFI\_CONNECTED** state.
- 

### 2. Wi-Fi Connected (WIFI\_CONNECTED):

- **Action:** Raspberry Pi is waiting for user input (via button press) to initiate communication.
  - **Transition:** On button press, transition to **SEARCHING\_AND\_COMMUNICATING** state.
- 

### 3. Searching and Communicating (SEARCHING\_AND\_COMMUNICATING):

- **Action:** Ping the ESP8266 to check its reachability. If reachable, initialize the socket and send a handshake message to initiate communication.
  - **Transition:** Once communication is established, transition to **START** state.
- 

### 4. Start Communication (START):

- **Action:** Raspberry Pi receives sensor data from the ESP8266 and processes it. Depending on the sensor intensity, different LEDs are activated.
- **Transition:** If data reception fails, move to **ERROR** state.

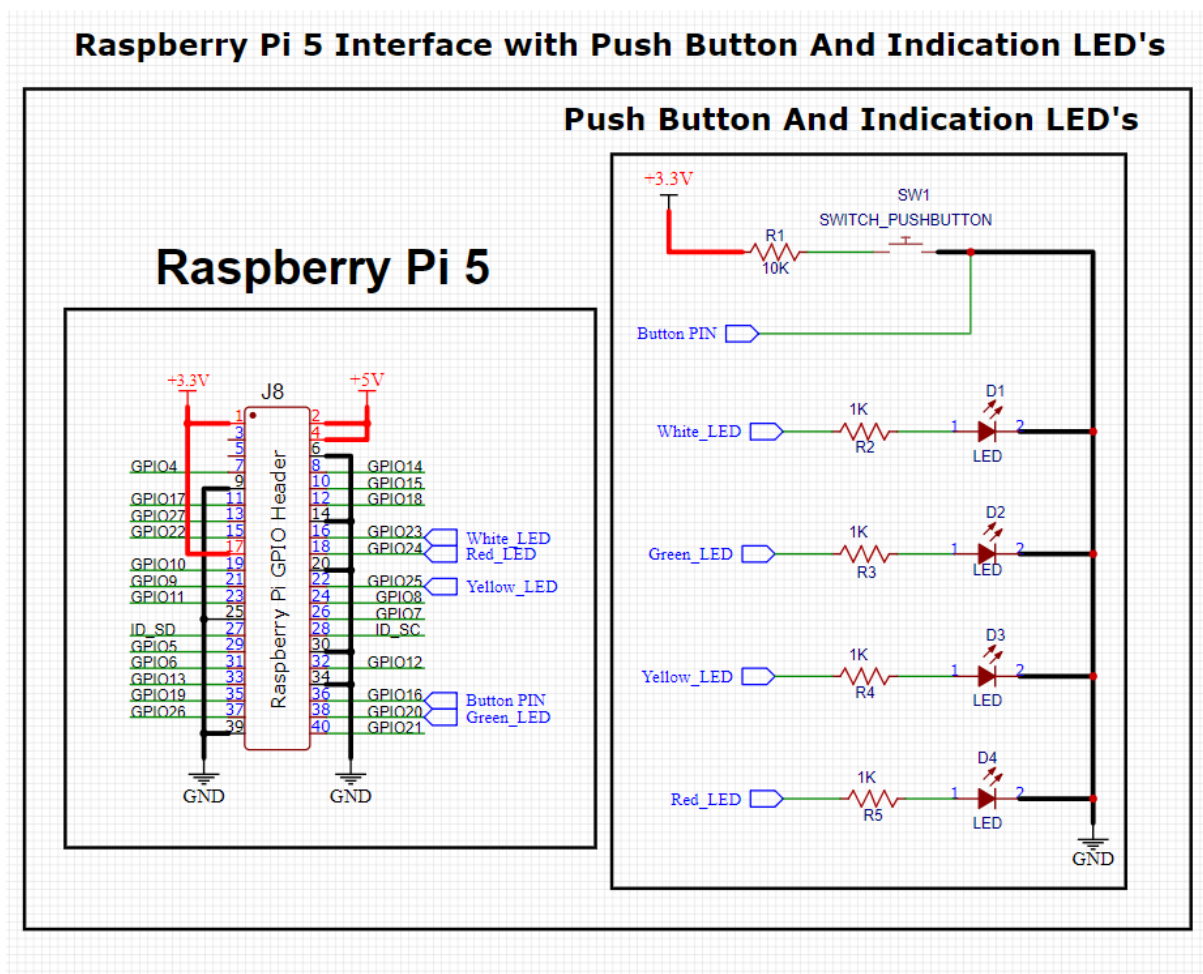
## 5. Error Handling (ERROR):

- **Action:** White LED blinks to indicate an error, waiting for user to press the button to reinitiate the system.
- **Transition:** On button press, reset system to **STOP** state.

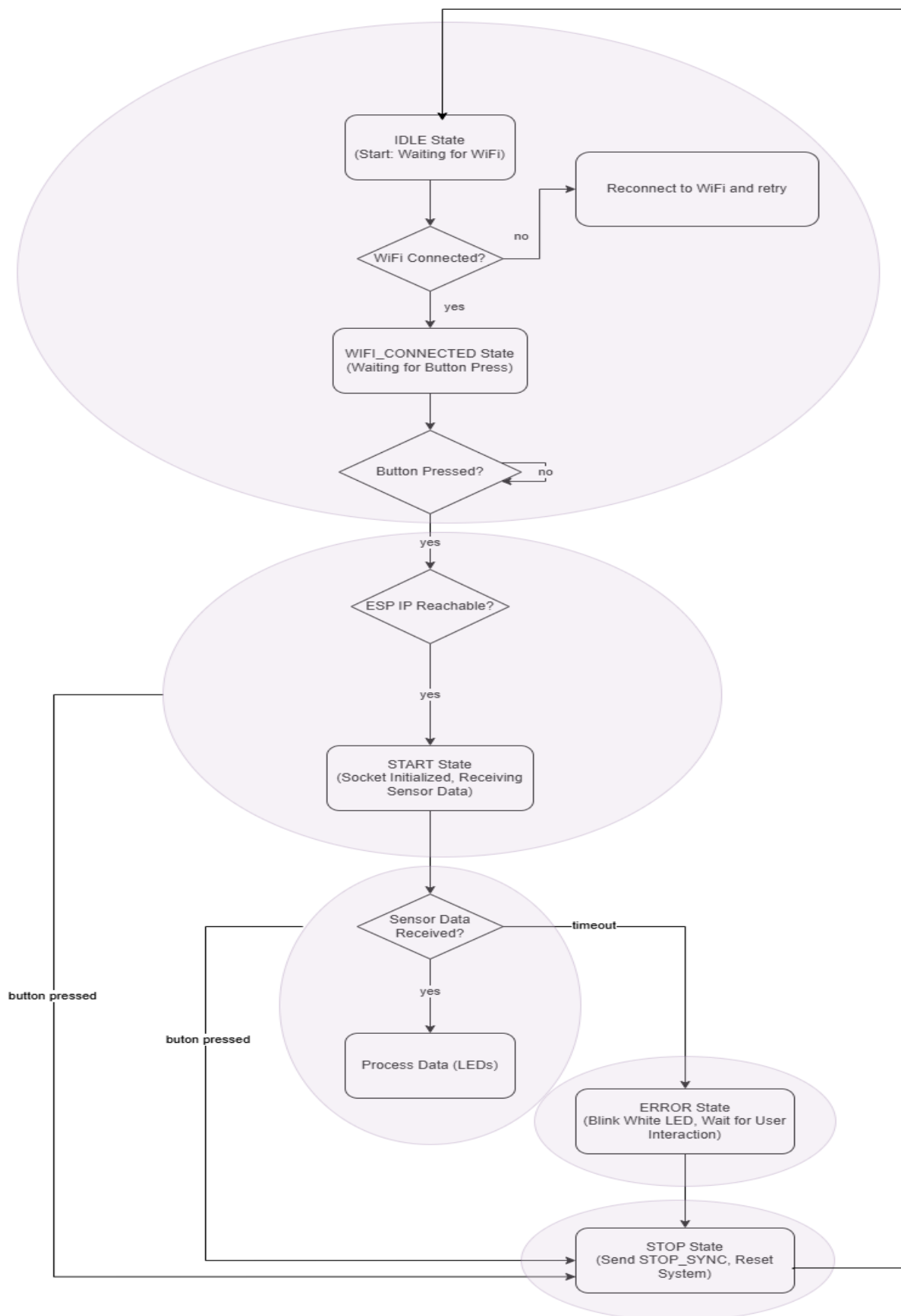
## 6. Stop Communication (STOP):

- **Action:** Stop communication, reset LEDs, deactivate the socket, and return to IDLE.

Schematic of the Raspberry PI connections:



## Graphical Flowchart for Raspberry Pi Program:



# Part 2- ESP8266 Wi-Fi Setup and Packet Delivery

## ESP8266 Code Functionality

- **Input:** Receive commands (**START\_SYNC**, **STOP\_SYNC**) from Raspberry Pi, collect sensor data.
- **Process:** Read sensor data, package it for transmission, handle the authentication of the communication establishment messages.
- **Output:** Send sensor data to Raspberry Pi over UDP.

## Flowchart for ESP8266 Program:

### 1. Start

- **Start Point:** Initializes Wi-Fi and starts UDP communication.
- Checks for Wi-Fi connection and prints IP address.

**Next**

Proceed to the **IDLE** State

---

### 2. IDLE State

- **Description:** The device is waiting for a handshake signal from the Raspberry Pi.
- **Condition:** If a valid configuration message (**START\_SYNC**) is received.

**Next**

**True:** Proceed to next state (**Handshake Done**).

**False:** Stay in IDLE and wait.

---

### 3. HANDSHAKE\_DONE State

- **Description:** After receiving **START\_SYNC**, the system is configured for data transmission.
- **Action:** Call **system\_Configuration()** to set up data collection.

**Next:**

Move to **SENDING\_DATA** state.

---

#### 4. SENDING\_DATA State

- **Description:** The device starts reading sensor data and sending it to the Raspberry Pi.
- **Action:** Activate timers for LED blinking, LDR reading, and sending averaged data through UDP.

**Next:**

If a **STOP\_SYNC** message is received, move to **STOP State** and reset all the configuration of the targeted device (Raspberry PI) saved

**Otherwise:** Stay in **SENDING\_DATA** and continue doing it work.

---

#### 5. STOP State

- **Description:** Device stops data collection and resets configuration.
- **Action:** Call **Target\_deConfiguration()** to reset device settings, stop timers, and go back to **IDLE**.

**Next:**

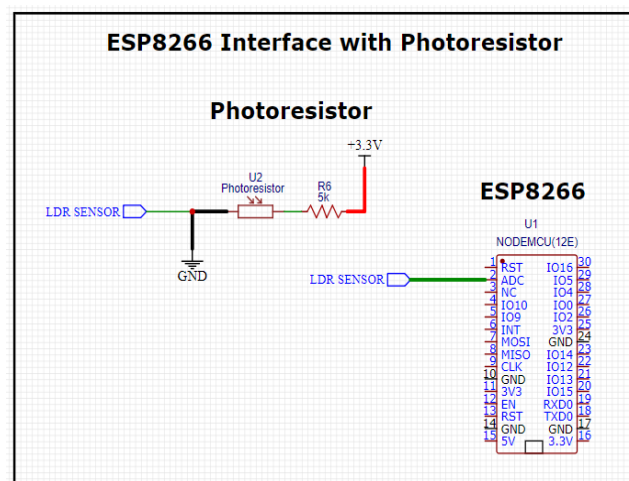
go back to **IDLE**.

---

**The main functions Processes (timers):**

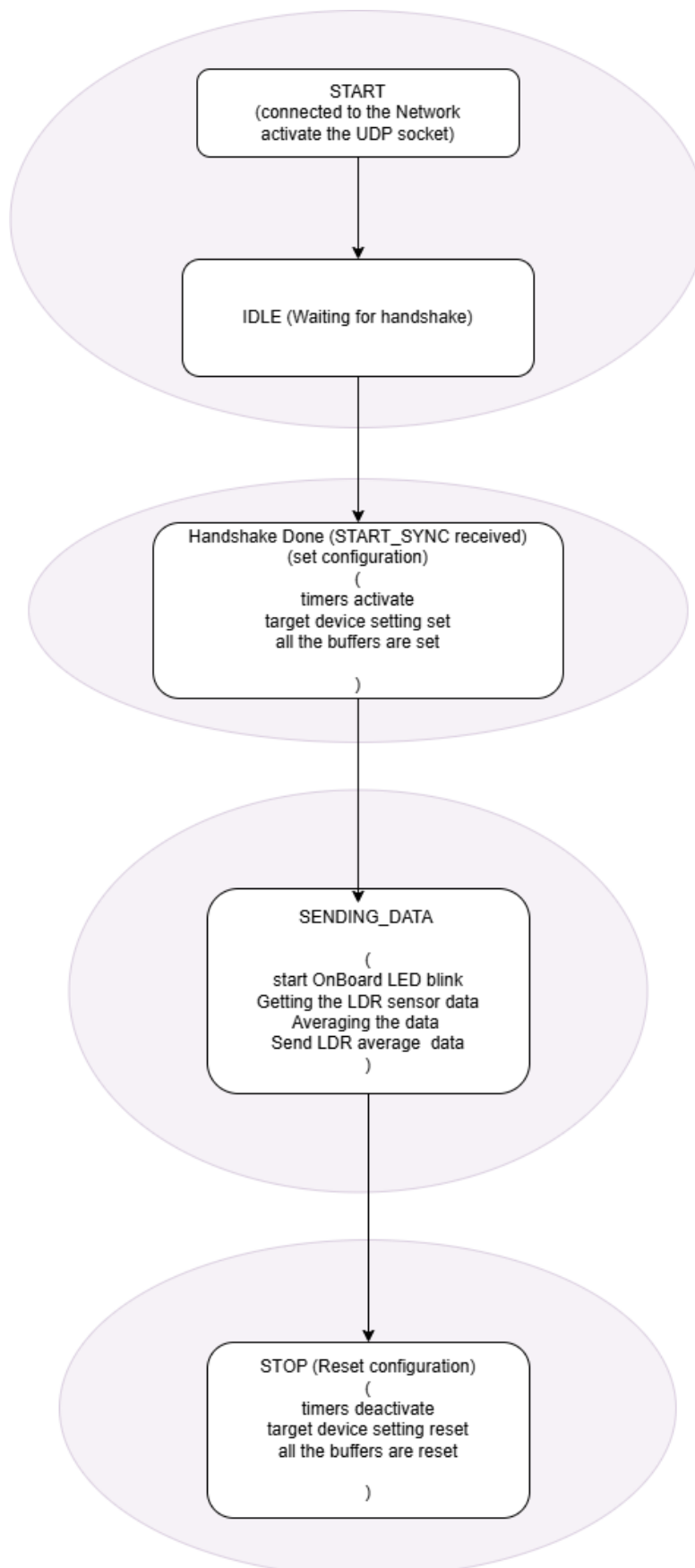
- **Blinking the on-board LED (LDR):**
  - Periodically Blinks the Led, to indicate that I am collecting the data and sending.
- **Reading Sensor Data (LDR):**
  - Periodically read LDR sensor values.
  - Store values in a buffer and calculate an average after 5 readings.
- **Sending UDP Data:**
  - Periodically send the averaged LDR readings via UDP to the Raspberry Pi.

Schematic of the Raspberry PI connections:





## Graphical Flowchart for ESP8266 Program:

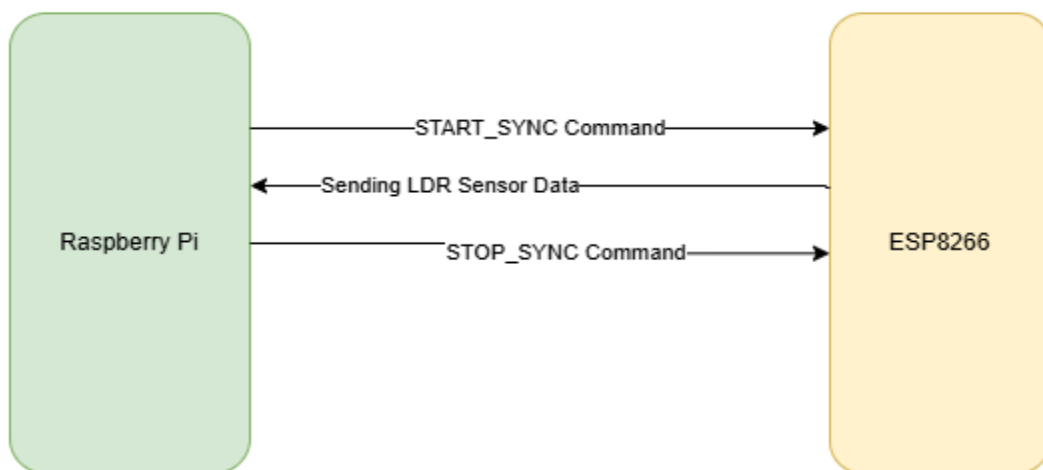


## Part 3- Packet Transfer Flow

The flowchart for packet transfer between the **Raspberry Pi** and **ESP8266** should show the communication flow between these two devices during various stages like configuration, data transmission, and stopping the communication. This will primarily focus on the handshake, sending sensor data, and stopping the sync process.

### Overview of Packet Transfer Flow:

1. **Raspberry Pi → ESP8266:**
  - Raspberry Pi sends a handshake (START\_SYNC) command to ESP8266.
  - If successful, ESP8266 configures itself to begin data transmission.
2. **ESP8266 → Raspberry Pi:**
  - ESP8266 starts sending sensor data (LDR readings) back to the Raspberry Pi over UDP.
3. **Raspberry Pi → ESP8266:**
  - Raspberry Pi sends a STOP\_SYNC command when it wants to stop receiving data.
  - ESP8266 deconfigures and resets the settings itself. And back to IDLE.



The image contains two circuit diagrams. The left diagram, titled 'Raspberry Pi 5 Interface with Push Button And Indication LED's', shows a Raspberry Pi 5 with its GPIO pins connected to a push button (SW1) and four LEDs (White\_LED, Green\_LED, Yellow\_LED, Red\_LED). The push button is connected to +3.3V through a 10K resistor (R1) and to a GPIO pin (GPIO14) through a 10K resistor (R2). The LEDs are connected to +3.3V through resistors (R2, R3, R4, R5) and to GPIO pins (GPIO14, GPIO15, GPIO16, GPIO17). The right diagram, titled 'ESP8266 Interface with Photoresistor', shows an ESP8266 module connected to a photoresistor (U2) and a 5V supply. The photoresistor is connected to +5V through a 5K resistor (R6) and to a GPIO pin (GPIO14) through a 5K resistor (R7). The ESP8266 module is shown with its pins connected to +5V, GND, and a 3.3V supply.

## 1. Raspberry Pi 5 Interface with Push Button and LED Indicators

- The **General-Purpose Input/Output (GPIO)** pins on the **Raspberry Pi 5** are used to interface with external components like push buttons and LED indicators.
- The Raspberry Pi's GPIO header (J8) has multiple pins, including **3.3V**, **5V**, **GND**, and several programmable input/output pins. These GPIO pins allow the Raspberry Pi to receive signals (e.g., from a push button) or send signals (e.g., to LEDs to turn them on or off).

- **Push Button (SW1):**
    - A **push button** is connected to one of the GPIO pins of the Raspberry Pi.
    - **One side of the push button** is connected to the GPIO pin via a **10kΩ resistor (R1)**, acting as a pull-down resistor. This ensures that when the button is not pressed, the GPIO pin reads a **LOW** signal (0V).
    - **The other side of the button** is connected to **3.3V**. When the button is pressed, the GPIO pin is pulled high (3.3V), and the Raspberry Pi can detect this state change as a **button press**.
  - **LED Indicators:**
    - Four **LEDs** are connected to separate GPIO pins, allowing the Raspberry Pi to control each LED individually by sending signals through these pins.
    - Each LED has a **1kΩ current-limiting resistor** connected in series with it to protect the LED and the Raspberry Pi's GPIO pins from excessive current.
    - The following LEDs are connected:
      1. **White LED** via resistor **R2**
      2. **Green LED** via resistor **R3**
      3. **Yellow LED** via resistor **R4**
      4. **Red LED** via resistor **R5**
    - The Raspberry Pi can turn these LEDs on or off by toggling the GPIO pin to **HIGH** (3.3V) or **LOW** (0V).
  - **Power and Ground:**
    - The **3.3V** and **5V** pins on the Raspberry Pi provide power to components that need it, like sensors or circuits.
    - All components (push button and LEDs) are grounded via the **GND pin** of the Raspberry Pi, ensuring a complete electrical circuit for proper functioning.
- 

## 2. ESP8266 Interface with LDR Sensor

- **LDR (Photoresistor) Sensor:**
  - The **Light Dependent Resistor (LDR)** is used to measure light intensity. Its resistance varies based on the amount of light falling on it — the more light, the lower its resistance, and vice versa.
  - This change in resistance is used to create a **voltage divider circuit** with a fixed resistor.

- **Voltage Divider Circuit:**
  - **One side of the LDR** is connected to **+3.3V**, which provides power to the sensor.
  - **The other side of the LDR** is connected to **GND** through a **5kΩ resistor (R6)**. This forms a voltage divider, which means that the voltage at the point between the LDR and the resistor changes depending on the light intensity sensed by the LDR.
  - As the LDR's resistance changes with light intensity, the voltage at the point between the LDR and the resistor also changes. This voltage is proportional to the light intensity.
- **ADC Pin on ESP8266:**
  - The **Analog-to-Digital Converter (ADC)** pin of the **ESP8266** (Pin **A0**) is connected to the point between the LDR and the 5kΩ resistor. The ADC pin reads the varying voltage from the voltage divider circuit.
  - The ESP8266 has only **one ADC pin (A0)**, which can measure voltages between 0V and 1V. Any higher voltage needs to be scaled down appropriately (usually done with resistors or a voltage divider).
  - The ESP8266 uses the analog voltage signal from the LDR to interpret the current light level. This information can then be processed, transmitted over Wi-Fi, or used to control other devices.